

AI News Influencer - Project Definition

Sundai MIT IAP 2026

Project Overview

An autonomous agentic system that monitors X (Twitter) for relevant news, generates engaging LinkedIn content with images, posts automatically, responds to comments, and continuously improves its strategy based on engagement metrics.

Goal: Build a practical AI agent that replaces manual social media management while teaching modern AI development patterns.

Success Metrics

- System posts 1/day for 1 week without manual intervention
 - 80%+ comment response rate
 - Measurable engagement improvement over baseline
-

Prebuild

All teaching team members must build working version before teaching.

Suggested Development Blocks:

- 1: Scraping + Storage
- 2: Agent Logic + Content Gen
- 3: LinkedIn Integration + Image Gen
- 4: Dashboard + Metrics
- 5: RAG + Memory System

Timeline: 2-3 weeks before IAP starts

User Requirements

Primary Users: Content creators, researchers, professionals managing LinkedIn presence

Core Needs:

- Autonomous news discovery and content creation
 - Branded, consistent posting style
 - Audience engagement without manual intervention
 - Performance tracking and strategy optimization
 - Control over what gets posted (approval/scheduling dashboard)
-

Functional Requirements

Content Generation

- Search X accounts for domain-specific news (AI/research/tech)
- Filter and rank content by relevance and engagement potential
- Generate LinkedIn posts with appropriate tone and hashtags
- Create contextual images using AI image generation
- Tag relevant people/organizations

Audience Interaction

- Monitor comments on posts
- Generate contextually appropriate responses
- Maintain conversation threads

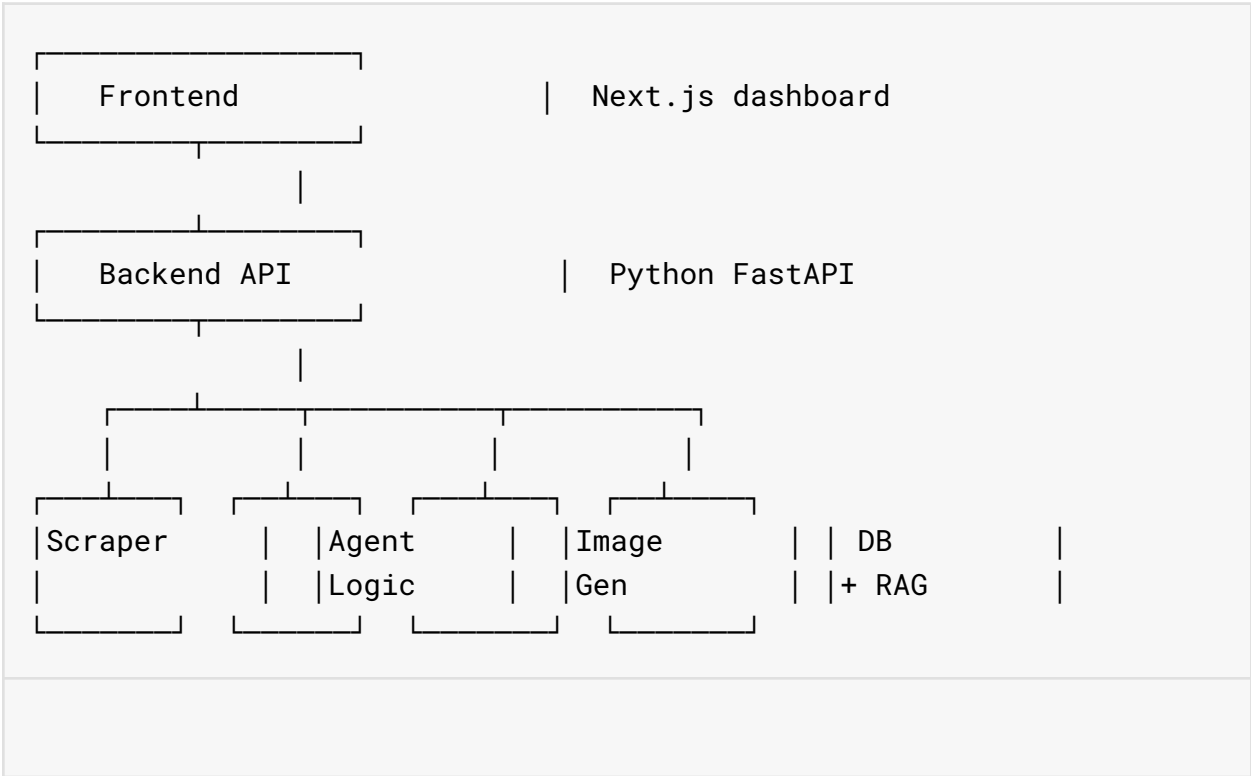
Performance Optimization

- Track engagement metrics (likes, comments, shares, views)
- Log all actions and decisions
- Adapt posting strategy based on performance data
- A/B test different content styles

User Interface

- Dashboard showing scheduled posts, metrics, performance trends
 - Manual override and editing capabilities
 - System configuration (posting frequency, topics, tone)
-

System Architecture



Tech Stack

Component	Technology	Rationale
Backend	Python 3.11+	AI/ML ecosystem, LangChain/LlamaIndex
Frontend	Next.js 14	Modern React framework, easy deployment
Database	SQLite + Vector extension	Simple, portable, built-in RAG
LLM	Claude API (Anthropic)	Strong reasoning, long context
Image Gen	DALL-E 3 or Stable Diffusion	Quality vs. cost tradeoff

Scraping	Playwright + BeautifulSoup	Handle dynamic content
API Framework	FastAPI	Async, type-safe, auto docs
Hosting	Digital Ocean Droplet	Simple, no AWS complexity
Validation	Pydantic v2	Type safety, data validation
IDE	VS Code + Claude Code	AI-assisted development

Course Timeline (5 Days)

Day 1: Foundation & Scraping

Goal: Basic infrastructure + data collection

Tasks:

- Set up project structure (backend/frontend repos)
- Initialize SQLite database with schema
- Build X.com scraper (Playwright)
- Store scraped content with metadata
- Basic CLI to test scraping

Deliverable: Working scraper that stores tweets in DB

Day 2: Agent Logic & Content Selection

Goal: Intelligence layer for choosing what to post

Tasks:

- Design agentic architecture (tool calling with Pydantic)
- Implement content ranking algorithm
- Build post generator with system prompts
- Create prompt templates for different content types
- Test content generation pipeline

Deliverable: Agent that selects and generates LinkedIn posts from scraped data

Day 3: Image Generation & LinkedIn Integration

Goal: Multimodal content + posting capability

Tasks:

- Integrate image generation API
- Build image prompt engineering logic
- Implement LinkedIn API posting
- Add hashtag and tagging logic
- Test end-to-end posting flow

Deliverable: System that posts text + images to LinkedIn

Day 4: Engagement & Memory

Goal: Interaction and knowledge retention

Tasks:

- Implement comment monitoring
- Build response generation agent
- Create RAG system for post history
- Add vector embeddings for semantic search
- Implement memory/context management

Deliverable: Bot that responds to comments using past context

Day 5: Metrics & Self-Improvement

Goal: Monitoring and optimization loop

Tasks:

- Build metrics collection (LinkedIn API + scraping)

- Create logging system for agent decisions
- Implement strategy adjustment logic
- Build Next.js dashboard (metrics visualization)
- Deploy to Digital Ocean

Deliverable: Full system with dashboard and auto-optimization

Lectures

that teach for & alongside development timeline

Lecture 0: Claude Code & AI-Assisted Development (Throughout)

- Using Claude Code effectively
- Creating custom skills
- Debugging with AI
- Code review best practices

Lecture 1: Agentic Systems & Tool Calling (Day 1)

- What are AI agents vs. chatbots
- Tool calling with Pydantic
- Orchestration patterns (Chain of Thought, ReAct)
- Claude API basics and prompt engineering
- (optional) Models and which models to use

Lecture 2: Web Scraping & Data Storage (Day 1)

- Rate limiting and ethical scraping
- SQLite schema design
- Playwright for dynamic sites?
- Data cleaning and normalization

Lecture 3: RAG & Vector Databases (Day 2)

- Embeddings fundamentals
- Retrieval-augmented generation patterns
- Chunking strategies
- Vector search with SQLite (sqlite-vss)

Lecture 4: Multimodal AI (Day 3)

- Image generation (Stable Diffusion on Replicate)
- Prompt engineering for images
- Vision models for context understanding

Lecture 5: FastAPI & Backend Design (Days 2-3)

- API design & best practices
- Digital Ocean backend setup with auth
- Job scheduling (APScheduler)?
- Async Python patterns?

Lecture 6: Next.js Dashboard (Day 4-5)

- React Server Components
- API routes and data fetching
- Real-time updates (polling vs. WebSockets)
- Deployment (Vercel vs. self host on digital ocean?)
- Buying domain and linking?

Lecture 7: LLM Observability & Evals (Day 5)

- Logging LLM inputs/outputs
- Defining success metrics
- A/B testing for prompts
- Learning loops for system self improvement & continual learning

Missing Components to Address

High Priority

1. **Authentication & Security**
 - LinkedIn OAuth flow
 - API key management (environment variables)
2. **Error Handling & Resilience**
 - Retry logic for API failures
 - Fallback strategies (if image gen fails)
 - Queue system for posts (what if LinkedIn API is down? Welcome to Cloudflare anyone?)
3. **Content Approval Flow**
 - Review posts before they go live
 - Manual edit/reject capability
 - Scheduling system
4. **Compliance & Safety**
 - Guardrails & Content filtering (no harmful/offensive posts)
 - Attribution for sources
 - Rate limiting to avoid spam

Medium Priority

5. **Testing Strategy**
 - TDD (test driven development)
 - Unit tests for agent logic
6. **Configuration Management**
 - System prompts and how to improve
 - A/B test configuration

Nice to Have

8. **Computer Use (Optional Advanced) (Nate's gonna love this)**
 - Agent browses dashboard to analyze metrics
 - Self-adjusts configuration based on visual analysis